Probability Density Estimation & Maximum Likelihood Estimation

Probability Density is a concept in probability theory that is used to describe the likelihood of a continuous random variable taking on a specific value within a given range. It is represented by a **Probability Density Function (PDF)**, a mathematical function specifying how the probability of the variable is distributed across different values.

Probability Density Function (PDF)

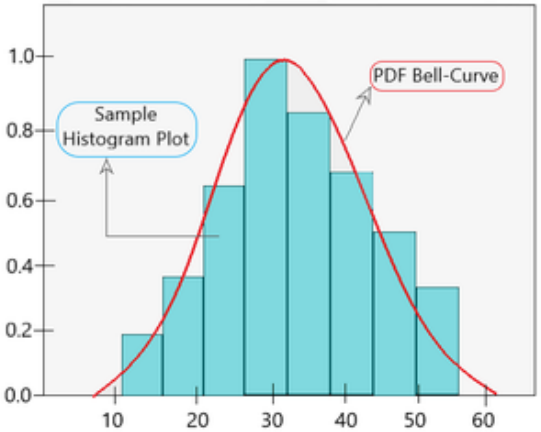
Probability Density Function (PDF) is a mathematical function that describes the likelihood of a continuous random variable from a sub-sample space falling within a particular range of values and not just one value.

$$P(a \leq X \leq b) = \int_a^b f(x)dx$$

Parametric Density Estimation

Parametric Density Estimation is a statistical technique used to estimate the probability distribution of a dataset by assuming that the data follows a specific distribution with a set of parameters.

A normal distribution has two given parameters, mean and standard deviation. We calculate the sample mean and standard deviation of the random sample taken from this population to estimate the density of the random sample. The reason it is termed as *'parametric'* is due to the fact that the relation between the observations and its probability can be different based on the values of the two parameters.

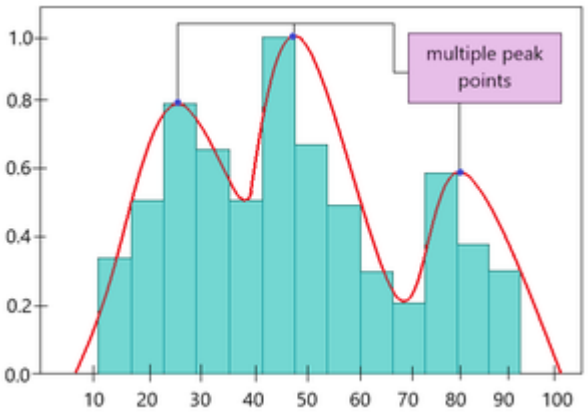


Non-parametric Density Estimation

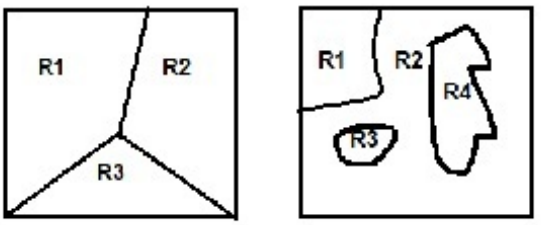
Non-Parametric Density Estimation is a statistical method used to estimate the probability distribution of a dataset without assuming that the data follows any specific parametric distribution

In some cases, the Probability Density Function may not fit the random sample as it doesn't follow a normal distribution (i.e instead of one peak there are multiple peaks in the graph). Here, instead of using distribution parameters like mean and standard deviation, a particular algorithm is used to estimate the probability distribution. Thus, it is known as a ***'nonparametric density estimation'*.**

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)$$



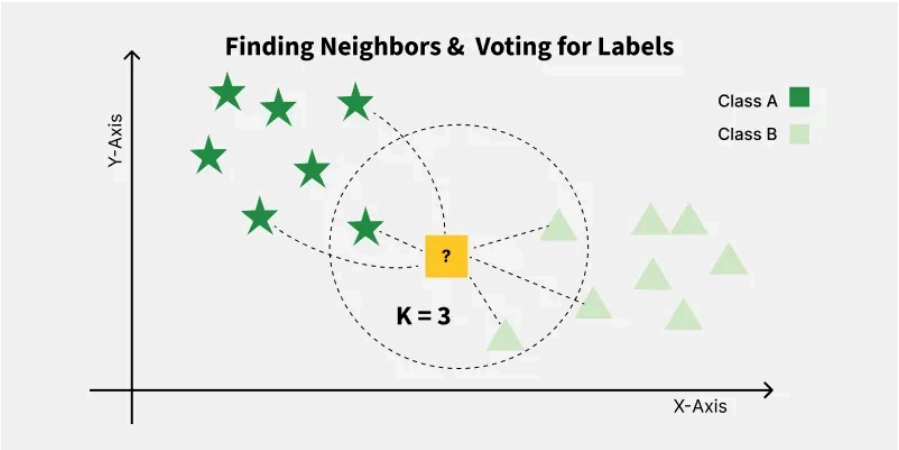
YOU KNow



Classifier and decision boundaries

K-Nearest Neighbor(KNN) Algorithm

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm generally used for classification but can also be used for regression tasks. It works by finding the "k" closest data points (neighbors) to a given input and makes a predictions based on the majority class (for classification) or the average value (for regression).



1. Euclidean Distance

Euclidean distance is defined as the straight-line distance between two points in a plane or space. You can think of it like the shortest path you would walk if you were to go directly from one point to another.

$$\text{distance}(x, X_i) = \sqrt{\sum_{j=1}^{d} (x_j - X_{i_j})^2]}$$

2. Manhattan Distance

This is the total distance you would travel if you could only move along horizontal and vertical lines like a grid or city streets. It's also called "taxicab distance" because a taxi can only drive along the grid-like streets of a city.

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

Feature Selection Techniques

Feature selection is a core step in preparing data for machine learning where the goal is to identify and keep only the input features that contribute most to accurate predictions. By focusing on the most relevant variables, feature selection helps build models that are simpler, faster, less prone to overfitting and easier to interpret especially when we use datasets containing many features, some of which may be irrelevant or redundant.

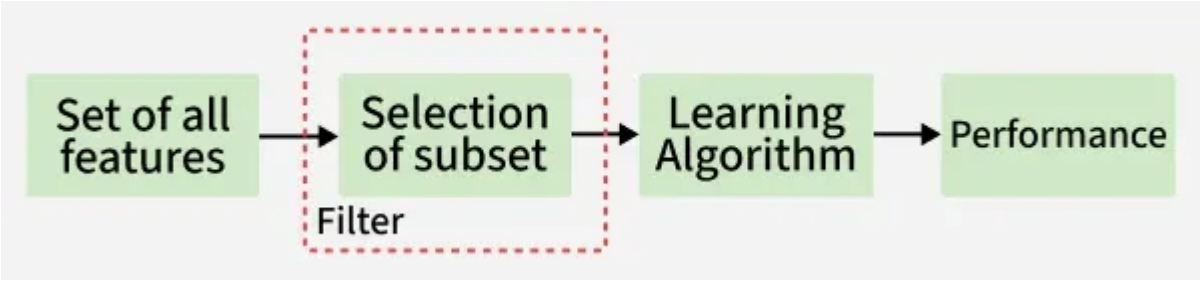
Need of Feature Selection

Feature selection methods are essential in data science and machine learning for several key reasons:

- **Improved Accuracy**: Focusing only on the most relevant features enables models to learn more effectively often resulting in higher predictive accuracy.
- **Faster Training**: With fewer features to process, models train more quickly and require less computational power hence saving time.
- **Greater Interpretability**: Reducing the number of features makes it easier to understand, analyze and explain how a model makes its decisions which is helpful for debugging and transparency.
- **Avoiding the Curse of Dimensionality**: Limiting feature count prevents models from being overwhelmed in high-dimensional spaces which helps in maintain performance and reliable results.

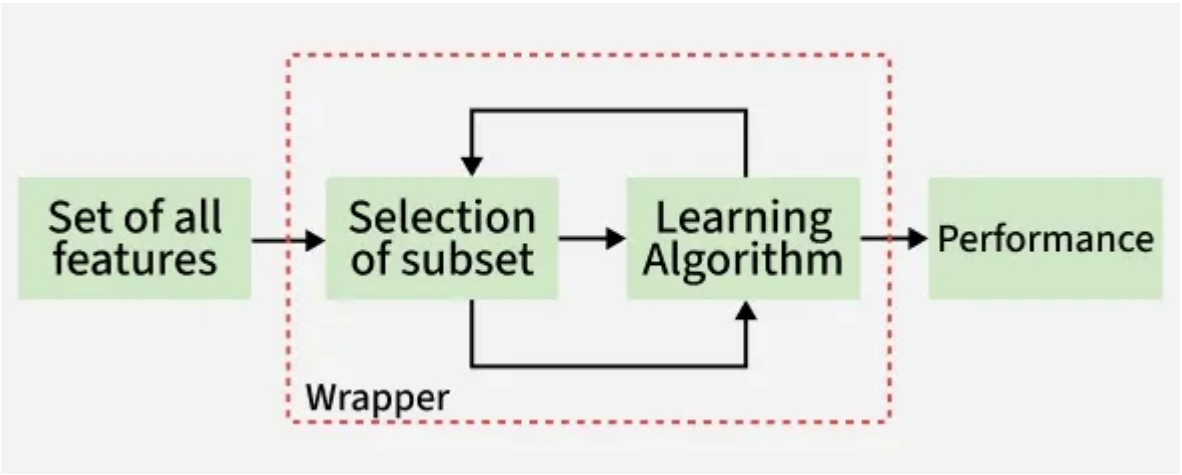
Types of Feature Selection Methods

Filter methods evaluate each feature independently with target variable. Feature with high correlation with target variable are selected as it means this feature has some relation and can help us in making predictions.



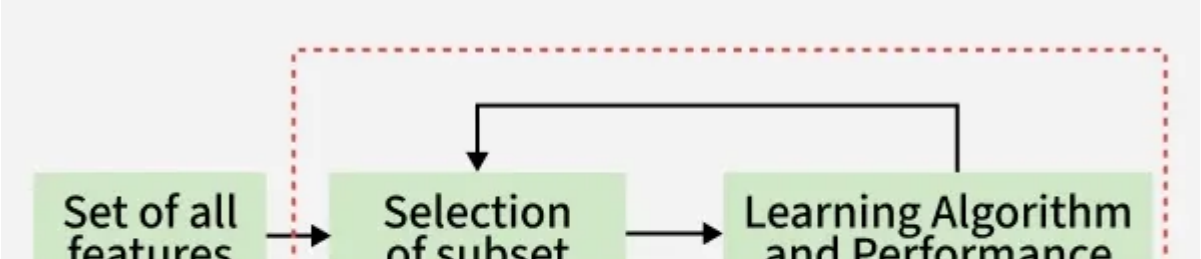
2. Wrapper methods

Wrapper methods are also referred as greedy algorithms that train algorithm. They use different combination of features and compute relation between these subset features and target variable and based on conclusion addition and removal of features are done. Stopping criteria for selecting the best subset are usually pre-defined by the person training the model such as when the performance of the model decreases or a specific number of features are achieved.



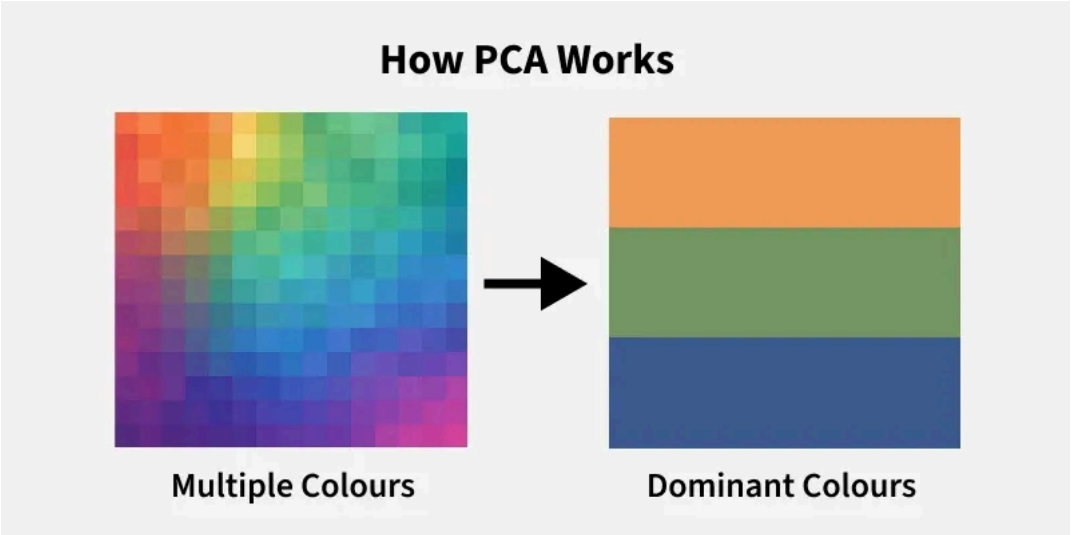
3. Embedded methods

Embedded methods perform feature selection during the model training process. They combine the benefits of both filter and wrapper methods. Feature selection is integrated into the model training allowing the model to select the most relevant features based on the training process dynamically.



Principal Component Analysis (PCA)

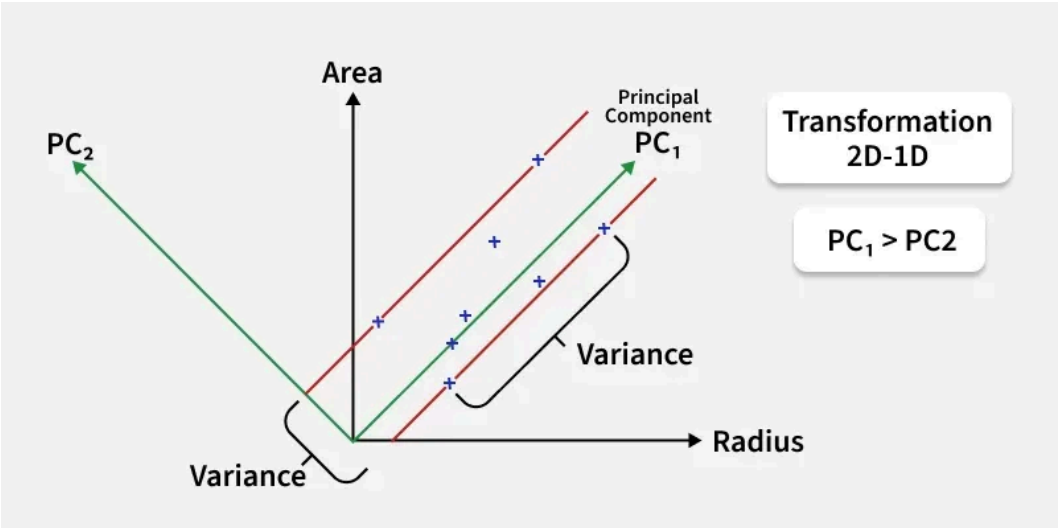
PCA (Principal Component Analysis) is a dimensionality reduction technique and helps us to reduce the number of features in a dataset while keeping the most important information. It changes complex datasets by transforming correlated features into a smaller set of uncorrelated components.



Step 4: Pick the Top Directions & Transform Data

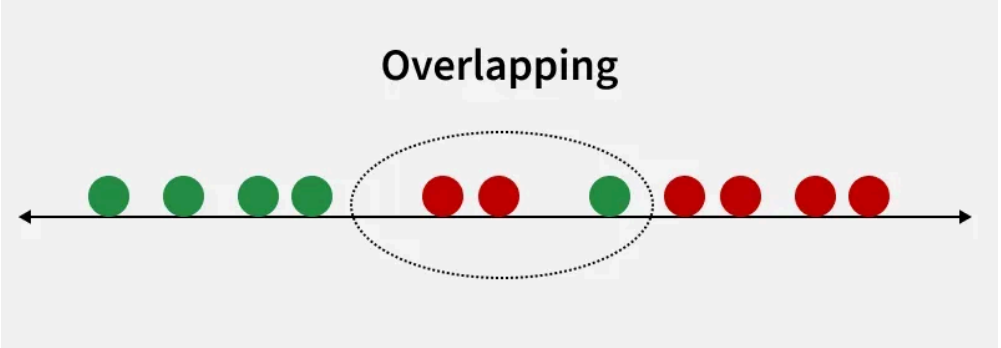
After calculating the eigenvalues and eigenvectors PCA ranks them by the amount of information they capture. We then:

1. Select the top k components that capture most of the variance like 95%.
2. Transform the original dataset by projecting it onto these top components.



Linear Discriminant Analysis

Linear Discriminant Analysis (LDA) also known as Normal Discriminant Analysis is supervised classification problem that helps separate two or more classes by converting higher-dimensional data space into a lower-dimensional space. It is used to identify a linear combination of features that best separates classes within a dataset.

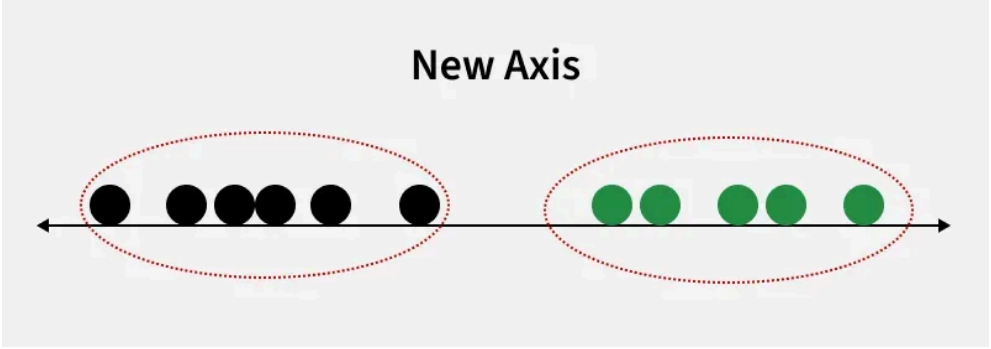


For example we have two classes that need to be separated efficiently. Each class may have multiple features and using a single feature to classify them may result in overlapping. To solve this LDA is used as it uses multiple features to improve classification accuracy. LDA works by some assumptions and we are required to understand them so that we have a better understanding of its working.

For LDA to perform effectively, certain assumptions are made:

- Gaussian Distribution: The data in each class should follow a normal bell-shaped distribution.
- Equal Covariance Matrices: All classes should have the same covariance structure.
- Linear Separability: The data should be separable using a straight line or plane.

LDA can produce very good results if it meets these assumptions. For example when data points belonging to two classes are plotted, if they are not linearly separable LDA will attempt to find a projection that maximizes class separability.



Feature Engineering for Pattern Recognition

1. What is Feature Engineering?

Feature Engineering is the process of **selecting, transforming, or creating features** from raw data so that machine learning or pattern recognition algorithms can classify or recognize patterns more accurately.

In simple words:
👉 *Convert raw data into meaningful inputs that help the model detect patterns.*

2. Why is Feature Engineering Important?

- Improves model accuracy
- Reduces noise and irrelevant information
- Helps models learn patterns clearly
- Reduces computational cost
- Makes classification easier and faster

3. Steps in Feature Engineering

(i) Feature Extraction

Extract useful information from raw data.

Examples:

- Image → edges, corners, textures
- Audio → pitch, MFCC
- Text → TF-IDF, word counts
- Signals → frequency components via FFT

(ii) Feature Selection

Choose only the most important features.

Methods:

- Filter methods → Correlation, Chi-Square
- Wrapper methods → Forward/Backward selection
- Embedded methods → Lasso, Decision Trees

(iii) Feature Transformation

Modify features to improve the representation.

Examples:

- Normalization (0 to 1 range)
- Standardization (mean = 0, variance = 1)
- PCA (dimensionality reduction)
- Log transforms to reduce skewness

4. Types of Features Used in Pattern Recognition

A. Statistical Features

- Mean
- Variance
- Standard deviation
- Skewness
- Kurtosis

Useful for: signals, sensors, time-series.

5. Dimensionality Reduction in Feature Engineering

Used to reduce the number of features while keeping important information.

Methods:

- **PCA (Principal Component Analysis)**
- **LDA (Linear Discriminant Analysis)**
- **t-SNE** (not usually for exam)

Benefits:

- Removes redundant features
- Reduces overfitting
- Makes computation faster

Difference Between Feature Selection and Feature Extraction

Feature selection and feature extraction are two key techniques used in machine learning to improve model performance by handling irrelevant or redundant features. While both works on data preprocessing, feature selection uses a subset of existing features whereas feature extraction transforms data into a new feature. In this article we will learn more about their key differences.

- Feature selection: involves selecting a **subset of the most relevant features that are actually contributing in prediction while discarding the rest features**. This helps improve reducing overfitting and increased accuracy. Common techniques include filter, wrapper and embedded methods.
- Feature extraction: transforms existing features into a new set of features that captures better underlying patterns in data. It is useful when raw data is in high dimension or complex. Techniques like PCA, LDA and Autoencoders are used for this purpose.

Difference Feature Selection and Feature Extraction Methods

Feature selection and feature extraction methods have their own advantages and disadvantages depending on the nature of the data and the task they handle.

Feature Selection	Feature Extraction
Selects a subset of relevant features from the original set of features.	Transforms original features into a new, more informative set.
Reduces dimensionality while keeping original features.	Reduces dimensionality by transforming data into a new space.

Methods include Filter, Wrapper and Embedded techniques.	Methods include PCA, LDA, Kernel PCA and Autoencoders.
Requires domain knowledge and feature engineering.	Can be applied to raw data without prior feature engineering.
Enhances interpretability and reduces overfitting.	Improves performance and handles nonlinear relationships.
May lose useful information if important features are removed.	May introduce redundancy and noise if extracted features are not well-defined.

CNN Neural Networks

https://www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/

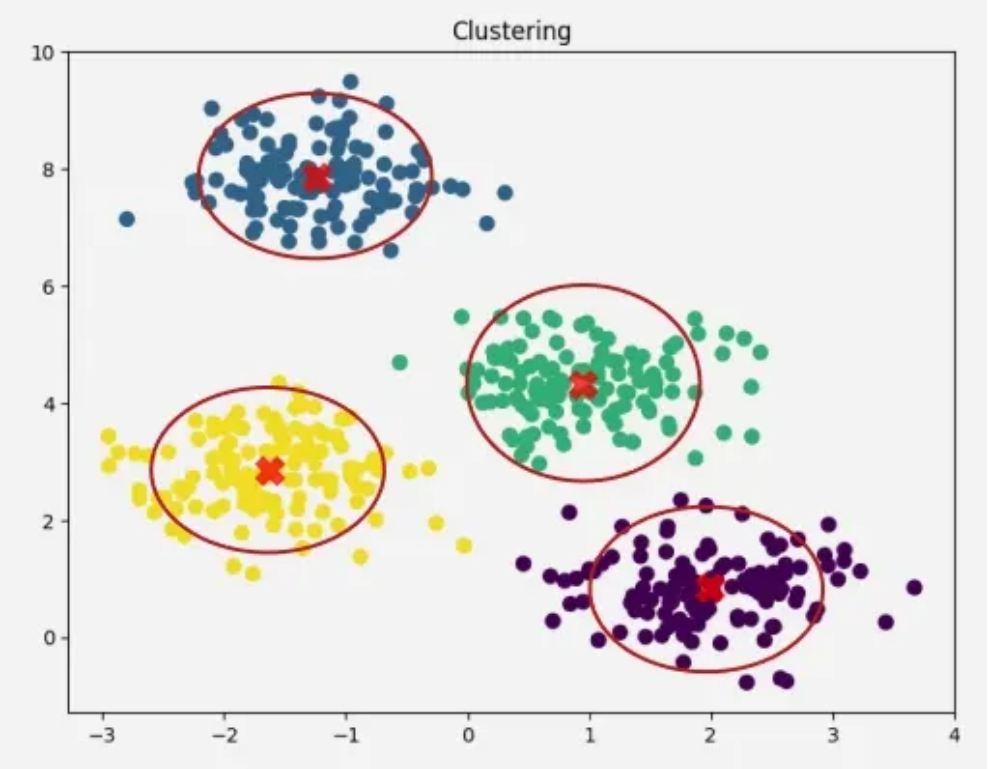
Introduction to Deep Learning

https://www.geeksforgeeks.org/deep-learning/types-of-neural-networks/

Clustering in Machine Learning

Clustering is an unsupervised machine learning technique that groups similar data points together into clusters based on their characteristics, without using any labeled data. The objective is to ensure that data points within the same cluster are more similar to each other than to those in different clusters, enabling the discovery of natural groupings and hidden patterns in complex datasets.

- **Goal**: Discover the natural grouping or structure in unlabeled data without predefined categories.
- **How**: Data points are assigned to clusters based on similarity or distance measures.
- **Similarity Measures**: Can include Euclidean distance, cosine similarity or other metrics depending on data type and clustering method.
- **Output**: Each group is assigned a cluster ID, representing shared characteristics within the cluster.



Types of Clustering

Let's see the types of clustering,

1. Hard Clustering: In hard clustering, each data point strictly belongs to exactly one cluster, no overlap is allowed. This approach assigns a clear membership, making it easier to interpret and use for definitive segmentation tasks.

- **Example**: If clustering customer data into 2 segments, each customer belongs fully to either Cluster 1 or Cluster 2 without partial memberships.
- **Use cases**: Market segmentation, customer grouping, document clustering.
- **Limitations**: Cannot represent ambiguity or overlap between groups; boundaries are crisp.

2. Soft Clustering: Soft clustering assigns each data point a probability or degree of membership to multiple clusters simultaneously, allowing data points to partially belong to several groups.

- **Example**: A data point may have a 70% membership in Cluster 1 and 30% in Cluster 2, reflecting uncertainty or overlap in group characteristics.
- **Use cases**: Situations with overlapping class boundaries, fuzzy categories like customer personas or medical diagnosis.
- **Benefits**: Captures ambiguity in data, models gradual transitions between cluster

Types of Clustering Methods

Clustering methods can be classified on the basis of how they form clusters,

1. Centroid-based Clustering (Partitioning Methods)

Centroid-based clustering organizes data points around central prototypes called centroids, where each cluster is represented by the mean (or medoid) of its members. The number of clusters is specified in advance and the algorithm allocates points to the nearest centroid, making this technique efficient for spherical and similarly sized clusters but sensitive to outliers and initialization.

2. Density-based Clustering (Model-based Methods)

Density-based clustering defines clusters as contiguous regions of high data density separated by areas of lower density. This approach can identify clusters of arbitrary shapes, handles noise well and does not require predefining the number of clusters, though its effectiveness depends on chosen density parameters.

Algorithms:

- DBSCAN (Density-Based Spatial Clustering of Applications with Noise): Groups points with sufficient neighbors; labels sparse points as noise.
- OPTICS (Ordering Points To Identify Clustering Structure): Extends DBSCAN to handle varying densities.

3. Connectivity-based Clustering (Hierarchical Clustering)

Connectivity-based (or hierarchical) clustering builds nested groupings of data by evaluating how data points are connected to their neighbors. It creates a dendrogram a tree-like structure that reflects relationships at various granularity levels and does not require specifying cluster numbers in advance, but can be computationally intensive.

Approaches:

- Agglomerative (Bottom-up): Start with each point as a cluster; iteratively merge closest clusters.
- Divisive (Top-down): Start with one cluster; iteratively split into smaller clusters.

Pros:

- Provides a full hierarchy, easy to visualize.
- No need to specify number of clusters upfront.

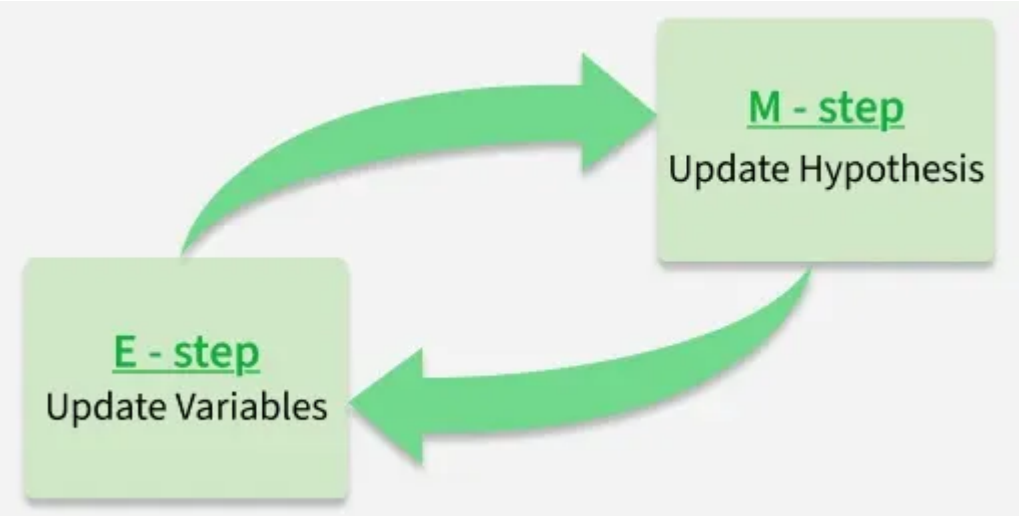
Cons:

- Computationally intensive for large datasets.
- Merging/splitting decisions are irreversible

Expectation-Maximization Algorithm - ML

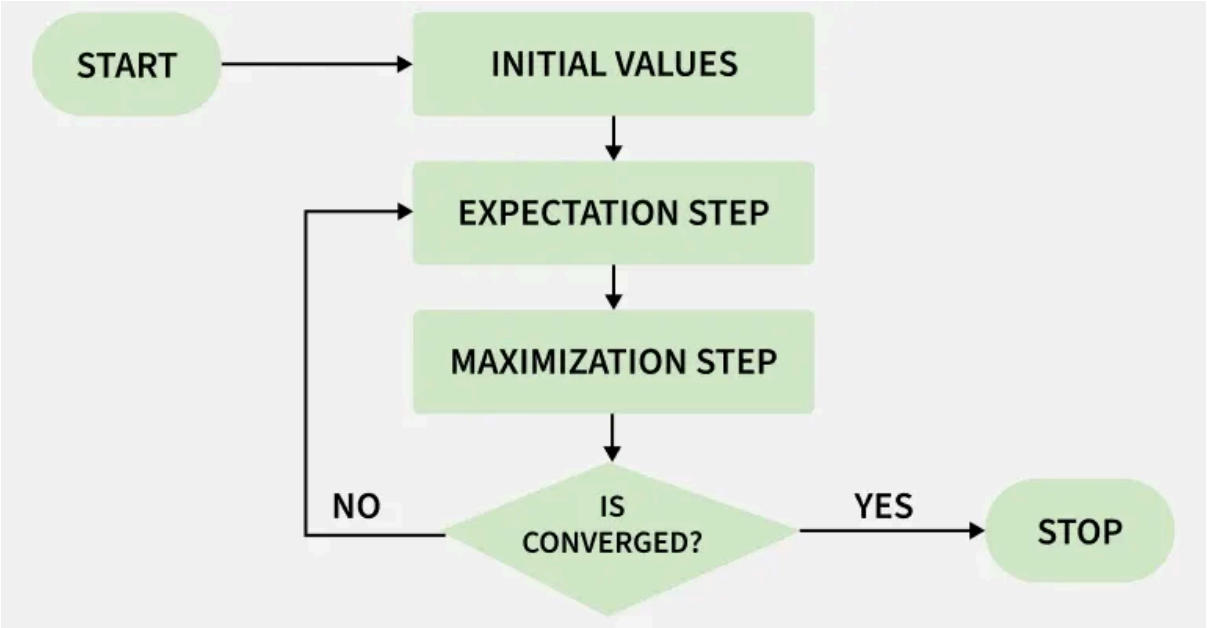
The Expectation-Maximization (EM) algorithm is a powerful iterative optimization technique used to estimate unknown parameters in probabilistic models, particularly when the data is incomplete, noisy or contains hidden (latent) variables. It works in two steps:

- **E-step (Expectation Step):** Using the current parameter estimates, the algorithm calculates the expected values of the missing or hidden variables. Essentially, it assigns probabilities or "responsibilities" to different hidden outcomes given the observed data.
- **M-step (Maximization Step):** With these updated expectations from the E-step, the algorithm then re-estimates the model parameters by maximizing the expected log-likelihood. This improves how well the model explains the observed data.



Working of Expectation-Maximization (EM) Algorithm

Here's a step-by-step breakdown of the process:



1. Initialization: The algorithm starts with initial parameter values and assumes the observed data comes from a specific model.

2. E-Step (Expectation Step):

- Find the missing or hidden data based on the current parameters.
- Calculate the posterior probability of each latent variable based on the observed data.
- Compute the log-likelihood of the observed data using the current parameter estimates.

3. M-Step (Maximization Step):

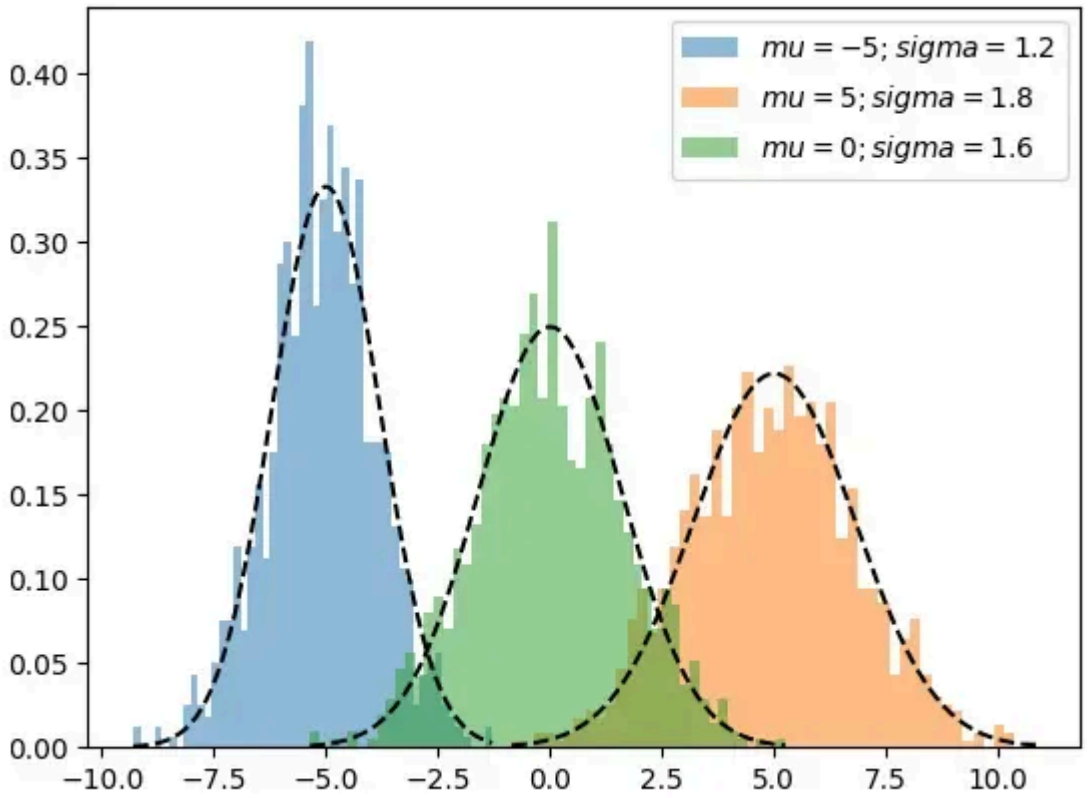
- Update the model parameters by maximize the log-likelihood.
- The better the model the higher this value.

4. Convergence:

- Check if the model parameters are stable and converging.
- If the changes in log-likelihood or parameters are below a set threshold, stop. If not repeat the E-step and M-step until convergence is reached

Gaussian Mixture Model

A Gaussian Mixture Model (GMM) is a probabilistic model that assumes data points are generated from a mixture of several Gaussian (normal) distributions with unknown parameters. Unlike hard clustering methods such as K-Means which assign each point to a single cluster based on the closest centroid, GMM performs soft clustering by assigning each point a probability of belonging to multiple clusters.



Working of GMM

Each cluster corresponds to a Gaussian distribution. For a given data point x_n of belonging to a cluster. GMM computes the probability it belongs to each cluster k:

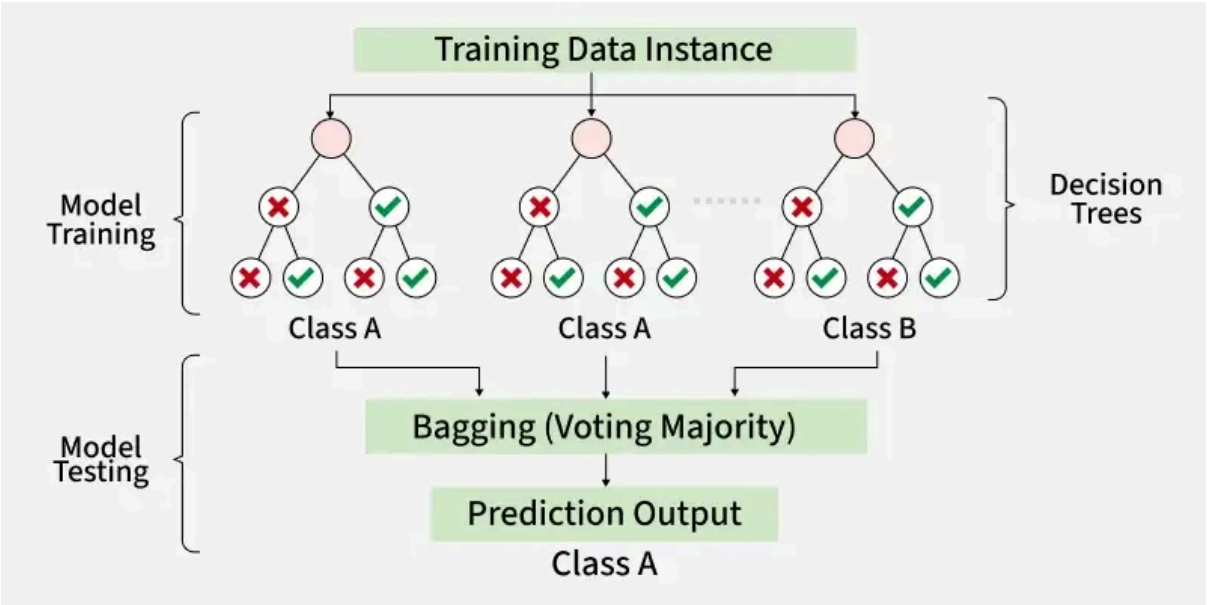
$$P(z_n = k \mid x_n) = \frac{\pi_k \cdot \mathcal{N}(x_n|\mu_k, \Sigma_k)}{\sum_{k=1}^{K} \pi_k \cdot \mathcal{N}(x_n|\mu_k, \Sigma_k)}$$

where:

- z_n=k is a latent variable indicating which Gaussian the point belongs to.
- π_k is the mixing probability of the k-th Gaussian.
- $\mathcal{N}(x_n \mid \mu_k, \Sigma_k)$is the Gaussian distribution with mean μ_k and covariance Σ_k

Random Forest Algorithm in Machine Learning

Random Forest is a machine learning algorithm that uses many decision trees to make better predictions. Each tree looks at different random parts of the data and their results are combined by voting for classification or averaging for regression which makes it as ensemble learning technique. This helps in improving accuracy and reducing errors.



Working of Random Forest Algorithm

- **Create Many Decision Trees:** The algorithm makes many decision trees each using a random part of the data. So every tree is a bit different.
- **Pick Random Features:** When building each tree it doesn't look at all the features (columns) at once. It picks a few at random to decide how to split the data. This helps the trees stay different from each other.
- **Each Tree Makes a Prediction:** Every tree gives its own answer or prediction based on what it learned from its part of the data.
- **Combine the Predictions:** For **classification** we choose a category as the final answer is the one that most trees agree on i.e majority voting and for **regression** we predict a number as the final answer is the average of all the trees predictions.

- **Why It Works Well:** Using random data and features for each tree helps avoid overfitting and makes the overall prediction more accurate and trustworthy.

Key Features of Random Forest

- **Handles Missing Data:** It can work even if some data is missing so you don't always need to fill in the gaps yourself.
- **Shows Feature Importance:** It tells you which features (columns) are most useful for making predictions which helps you understand your data better.
- **Works Well with Big and Complex Data:** It can handle large datasets with many features without slowing down or losing accuracy.
- **Used for Different Tasks:** You can use it for both **classification** like predicting types or labels and **regression** like predicting numbers or amounts.

Assumptions of Random Forest

- **Each tree makes its own decisions**: Every tree in the forest makes its own predictions without relying on others.
- **Random parts of the data are used**: Each tree is built using random samples and features to reduce mistakes.
- **Enough data is needed**: Sufficient data ensures the trees are different and learn unique patterns and variety.
- **Different predictions improve accuracy**: Combining the predictions from different trees leads to a more accurate final result.